

MODULE-15

AGILE METHODOLOGY

Agile Methodology

Agile Methodology is a way to manage projects by breaking them into smaller parts. It focuses on working together and making constant improvements. Teams plan, work on the project, and then review in a repeating cycle.

Agile Development Stages:

1. Planning
2. Requirement Analysis
3. Design
4. Implementation
5. Testing – Integration, System, User Acceptance and Performance Testings
6. Deployment
7. Review

Types of Agile Methodologies:

1. **Kanban:** Uses a visual workflow and limits work-in-progress (WIP) to manage continuous tasks and improve delivery speed.
2. **Scrum:** Works in short sprints with defined roles and meetings to deliver focused and incremental results.
3. **Extreme Programming (XP):** Focuses on strong coding practices, continuous feedback, and frequent releases for fast-changing requirements.

Popular Agile Tools: Jira, Trello, Asana, Pivotal Labs, ClickUp, Monday.com

Agile Manifesto

The Agile Manifesto is a foundational guide for modern software development.

It defines core values and principles that help teams adapt quickly and build high-quality products efficiently.

4 Core Values

1. **Individuals and Interactions** (over Processes and Tools): Prioritizes effective communication and teamwork over rigid tools and processes.
2. **Working Software** (over Comprehensive Documentation): Focuses on delivering functional software as the main measure of progress.
3. **Customer Collaboration** (over Contract Negotiation): Encourages continuous involvement of customers and stakeholders throughout development.
4. **Responding to Change** (over Following a Plan): Promotes adaptability to changing requirements, even in later changes of development.

12 Principles

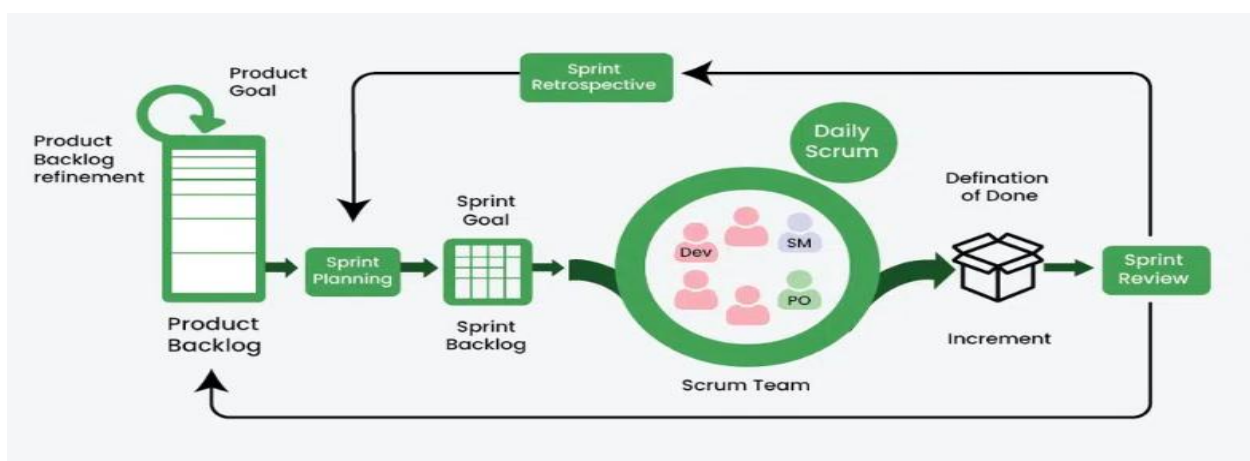
1. **Customer Satisfaction:** Deliver valuable software early and continuously to keep customers satisfied.
2. **Changing requirements:** Welcome and adapt to changing requirements, even in later stages of development.
3. **Frequent Delivery:** deliver working software frequently in shorter development lifecycles.
4. **Communicate Regularly:** ensure continuous collaboration between stakeholders and developers.
5. **Face-to-Face Communication:** Promote direct communication for better clarity and understanding.
6. **Measure work progress:** track progress based on working software rather than documentation.
7. **Sustainable Development Process:** maintain a consistent and sustainable pace of work.
8. **Good design:** focus on technical excellence and good design for better adaptability.
9. **Measure Progress:** Continuously evaluate performance to ensure effective outcomes.
10. **Support team members:** build projects around motivated individuals and provide them with support.
11. **Continuous Improvement** (Seeking Result): Improve processes regularly to achieve better results.
12. **Reflect and Adjust Regularly:** Reflect on team performance and make necessary improvements.

Aspect	Agile	Waterfall
Approach	Iterative and incremental	Sequential and linear
Planning	Flexible; plans evolve	Detailed upfront planning
Requirements	Can change throughout the project	Fixed before development begins
Delivery	Frequent releases in short iterations (sprints)	One final delivery at the end
Customer Involvement	Continuous feedback and collaboration	Limited, mainly at the beginning and end
Testing	Performed continuously during development	Done after development is complete
Documentation	Just enough documentation	Extensive documentation
Risk	Lower risk due to frequent feedback	Higher risk if requirements change
Best For	Dynamic, evolving projects	Stable, well-defined projects

SCRUM

Scrum is a widely used Agile framework for managing complex projects, especially in software development.

- Breaks work into small iterations called **sprints** for step-by-step delivery.
- Promotes collaboration, continuous feedback, and improvement.



Scrum Roles

1. **Scrum Master:** The **Scrum Master** is the facilitator and coach who ensures the Scrum process is followed correctly.
2. **Product Owner:** The **Product Owner** represents the customer and is responsible for maximizing the product's value.
3. **Development Team:** The **Development Team** consists of professionals who design, develop, test, and deliver the product.

Scrum Ceremonies

1. **Sprint Planning:** A meeting held at the beginning of every Sprint to decide **what work will be done** and **how it will be completed**.
 - Select backlog items for the Sprint.
 - Create the Sprint Goal.
 - Prepare the Sprint Backlog.
 - **Output: Sprint Backlog.**
2. **Daily Scrum (Daily Stand-up):** A 15-min daily meeting where the development team discusses progress.
 - track progress
 - identify issues early
 - coordinate team activities
3. **Sprint Review:** A meeting held at the end of the Sprint to demonstrate the completed product increment to stakeholders.
 - show completed features
 - collect stakeholder feedback
 - update the product backlog if needed
4. **Sprint Retrospective:** A meeting held after the Sprint Review to discuss how the team worked during the Sprint.
 - Identify what went well
 - Identify what could be improved
 - Plan improvements for the next sprint

Scrum Artifacts

1. **Product Backlog:** An ordered list of all features, bug fixes, improvements, and requirements for the product.
 - managed by the Product Owner
 - It is continuously updated
 - source of future Spring work

2. **Sprint Backlog:** The list of Product Backlog items selected for the current Sprint, along with tasks needed to complete them.
 - managed by development team
 - contains sprint goal
 - updated daily during the sprint
 - focuses only on the current sprint
3. **Increment:** The working product produced at the end of a Sprint that meets the Definition of Done.

Definition of Done (DoD)

The Definition of Done is a shared checklist that specifies when a Product Backlog item or Increment is considered complete.

Agile Estimation and Scrum Planning Techniques

Story Points

Story Points are a relative unit of estimation used in Scrum to measure the effort, complexity, and risk involved in completing a User Story.

They estimate how difficult the task is.

Planning Poker Technique

Planning Poker is a consensus-based estimation techniques used to assign Story Points to User Stories.

1. Product Owner explains the User Story.
2. Team discusses the requirements.
3. Each member secretly selects a Story Point card (1, 2, 3, 5, 8, 13...).
4. Everyone reveals their cards simultaneously.
5. If estimates differ, members explain their reasoning.
6. Repeat until the team agrees on a Story Point value.

This technique enhances collaboration. It prevents one person from influencing others.

Sprint Planning Process

Sprint Planning is the Scrum event held at the beginning of each Sprint to decide what work will be completed and how it will be done.

1. Product Owner presents the highest-priority Product Backlog items.
2. Team discusses requirements.
3. Team estimates effort (using Story Points/Planning Poker).
4. Team selects work based on its capacity.
5. Sprint Goal is defined.
6. Sprint Backlog is created.

Velocity

Velocity is the total number of Story Points completed by a team in one Sprint.

- Predicts how much work can be completed in future Sprints.
- Helps with Sprint planning.
- Measures team capacity.

Burndown Chart

A Burndown Chart is a graph that shows the remaining work versus time during a Sprint.

1. **X-axis:** Days of the Sprint
Y-axis: Remaining Story Points (or Tasks)
2. **Interpretation:**
 - Ideal Line: Expected progress
 - Actual Line: Team's actual progress
3. **Meaning:**
 - **Follows the ideal line:** Sprint is on track.
 - **Above the ideal line:** Team is behind schedule.
 - **Below the ideal line:** Team is ahead of schedule.

It Tracks Sprint progress, identifies delays early and helps the team adjust work during the Sprint.

Agile User Stories

User stories are a key component of agile software development. They are short, simple descriptions of a feature or functionality from the perspective of a user.

User Story refers to a short, informal, and simple description of software features that are required by the end-users in the software system.

Its main purpose is to provide software features that will add value to the customer requirements.

Pattern

As a [type of user], I want [an action], so that [some reason]

It follows the Role-Feature-Benefit Pattern

INVEST Principle

A good user story should be based on INVEST principle which expresses the quality of the user story because in base a good software product is completely dependent upon a good user story.

1. **Independent** -Not dependent on other.
2. **Negotiable** - Includes the important avoid contract.
3. **Valuable** - Provide value to customer.
4. **Estimable** - It should be estimated.
5. **Small** - It should be simple and small not complex.
6. **Testable** - It should be evaluated by pre-written acceptance criteria.

Acceptance Criteria

Acceptance Criteria are specific conditions that must be satisfied for a User Story to be considered complete and accepted by the Product Owner.

- Clarify requirements.
- Define expected behaviour.
- Reduce misunderstandings.
- Guide testing.

Given-When-Then Format

Acceptance Criteria are commonly written using the **Given–When–Then** format.

- **Given:** Initial condition
- **When:** Action performed
- **Then:** Expected result